

Reservoir Computing with the Human Connectome Project

Tebe Nigrelli
Giusi Scozzi
Antonio Perazza

Kevin Cesenj
Vecdi Gokmen
Emilio Vitali

Christina E. Christodoulou
Lorena Pavel
Angelo Spahiu

Following the kind direction of Postdoc Researcher Victor Buendia

Abstract

This project studies reservoir computing with human structural connectomes. We validate a standard pipeline on the Lorenz system, then replace the random recurrent matrix with a weighted Human Connectome Project adjacency matrix while training only a ridge-regression readout. Gaussian, connectome, leaky, and perturbed reservoirs are compared. All models recover bounded Lorenz-like dynamics but diverge quickly in exact trajectory prediction, consistent with chaos. Perturbations show node deletion is most damaging, while edge deletion and rewiring are weaker because much of the recurrent capacity and spectral scaling remains intact.

1. Introduction

The brain processes information through networks of neurons connected by synapses. Reservoir computing follows a related principle: a fixed recurrent network, the **reservoir**, transforms an input signal into a rich set of internal states, from which a simple linear readout is trained to make predictions. Since only the readout is trained, the method remains efficient while still modeling complex and chaotic signals.

Here we use structural connectivity data from the Human Connectome Project, which maps connections between brain regions. This lets us replace the usual random reservoir with a biologically grounded network that embeds brain structure directly in the model.

The reservoir is driven by a chaotic input signal and trained to predict the next time step. We then remove nodes and connections to simulate structural damage and measure how prediction

degrades. This tests how connectome topology supports computation and how robust it is to perturbations.

2. Reservoir dynamics

A reservoir can be understood through the analogy of a lake. Raindrops create ripples that spread, interact, and persist before fading; by observing the surface, one can infer something about the rain without seeing each drop. Similarly, an input signal perturbs the network, and that perturbation propagates through the connections and shapes later states. This intuition can be formalized by describing the reservoir as a dynamical system. The state of neuron i at time t , denoted by $r_{i(t)}$, evolves as

$$\frac{dr_i}{dt} = -r_i(t) + S \left(\sum_{j=1}^N W_{ij} r_j(t) \right)$$

The term $-r_{i(t)}$ represents decay, while the second term captures recurrent input through the weight matrix W after a nonlinear transformation. The current reservoir state is therefore a compressed representation of recent inputs.

In practice, its dynamics are approximated by evaluating the state at discrete time steps. The time derivative is expressed as

$$\frac{dr_i(t)}{dt} = \frac{r_i(t + \delta t) - r_i(t)}{\delta t}$$

which leads, via Euler discretization, to the update rule

$$r_i(t + \delta t) = r_i(t) + \delta t \frac{dr_i}{dt}$$

The richness of reservoir dynamics comes directly from its structure: an irregular, highly connected

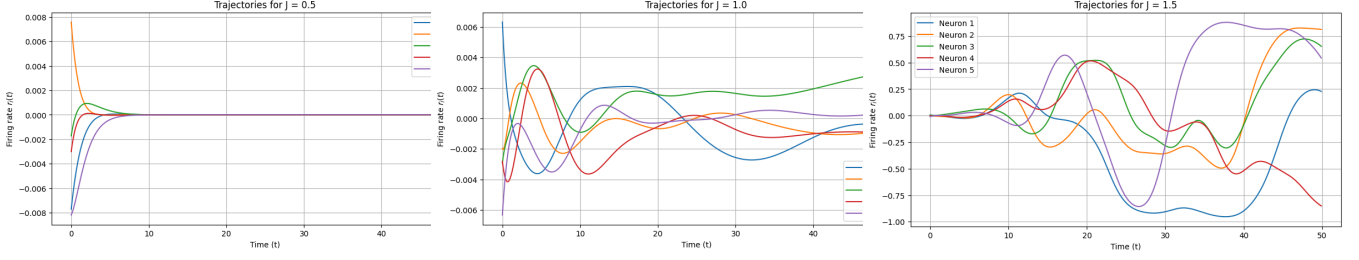


Figure 2: Neuron trajectories for $J < 1$, $J \approx 1$, and $J > 1$.

network turns even simple inputs into complex activity patterns. Because neurons respond differently, the network produces a diverse feature set from which the readout can extract information.

This balance is governed by the recurrent matrix W , which determines whether activity decays, persists, or grows. The resulting dynamical regime plays a central role in the reservoir's computational capacity.

To study this behavior in a controlled setting, we first use a Gaussian random reservoir, where the recurrent matrix W is built from independent Gaussian entries sampled as

$$W_{ij} \sim \mathcal{N}\left(0, \frac{J^2}{N}\right)$$

where N is the number of neurons and J controls the overall strength of recurrent interactions.

System stability can be understood through the eigenvalues of W . For large N , Girko's circular law places them approximately within a disc of radius J in the complex plane. The spectral radius, the magnitude of the largest eigenvalue, therefore determines whether perturbations decay or grow. In discrete time, stability requires eigenvalues inside the unit circle; in continuous time, it depends on their real parts.

Thus, three regimes appear: for $J < 1$, activity decays; for $J \approx 1$, the system is near criticality; and for $J > 1$, the dynamics become unstable or chaotic.

External input enters through the matrix W^{in} . In the state equation, it appears as $\sum_j W_{ij}^{\text{in}} h_j(t)$ and determines how the input affects each neuron before the nonlinearity. This fixed, randomly initialized matrix distributes the input across the

reservoir, so the state reflects both internal dynamics and incoming signals.

The goal is to use the reservoir state to predict the next input value, $h(t) \approx h(t + \delta t)$. The model maps $r(t)$ to the output through a linear transformation: $h(t) = W_{\text{readout}} * r(t)$, where W_{readout} contains the readout weights.

The internal and input weights remain fixed; only the readout weights are adjusted during training to approximate the next-step signal, $h(t) \approx h(t + \delta t)$.

The reservoir's internal structure is therefore not trained. Its computational power comes from fixed connections and their induced dynamics rather than from adapting recurrent weights during learning.

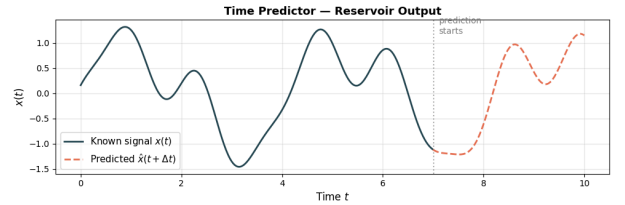


Figure 1: Reservoir time-series prediction example.

3. Dynamics overview

Building on the framework introduced above, we now investigate how the reservoir dynamics change as the coupling parameter J is varied.

Figure 2 shows representative neuron trajectories for different values of J . For $J < 1$, trajectories quickly converge to zero, so the system loses dependence on its initial state. Near the critical value, activity decays more slowly and retains information longer. For $J > 1$, trajectories become irregular and highly variable, indicating a transition to chaos.

To quantify this transition, we measure temporal variability. Networks of $N = 400$ neurons are simulated for $T = 50$ time units with step size $dt = 0.1$. For each J , the variance of each neuron’s activity is computed over time and averaged across neurons and 10 fixed-seed realizations.

The results are shown in Fig. 3. For small J , the mean temporal variance stays near zero, confirming rapid convergence. Beyond the critical regime, the variance rises sharply, reflecting sustained fluctuations.

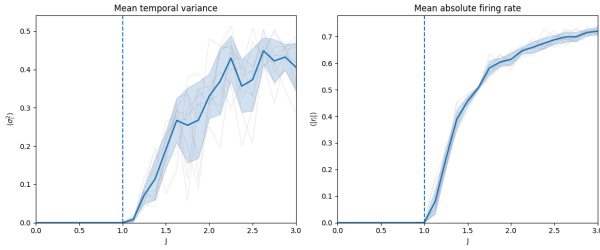


Figure 3: Mean temporal variance vs. J (10 trials, seed 42).

These regimes directly affect reservoir memory. In the stable regime ($J < 1$), the system quickly forgets past inputs as trajectories collapse to a fixed point. Near criticality ($J \approx 1$), slower decay improves memory. For $J > 1$, sensitivity to perturbations breaks consistent behavior and violates the Echo State Property.

The same transition appears at the population level. In Fig. 4, activation distributions broaden as J increases, shifting from a narrow concentration around zero to a wide, heterogeneous spread.

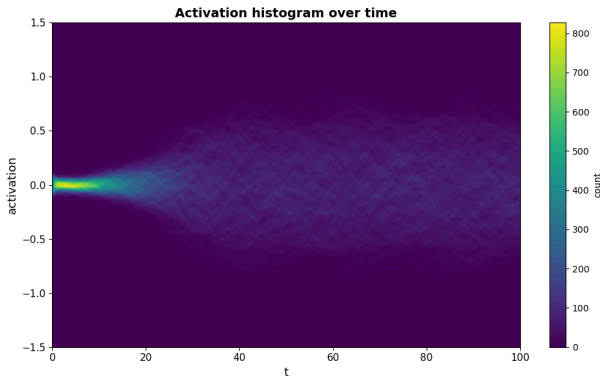


Figure 4: Activation histograms over time ($N = 500$, $T = 100$, $dt = 0.1$).

4. Training process

Before presenting the results, we outline the implementation strategy. The project has two phases. First, we built and validated the predictive pipeline with standard Gaussian random networks. After confirming that the baseline could model the Lorenz attractor in a controlled setting, we replaced the random matrices with empirical human connectome data to test the same task on a brain-derived topology.

In reservoir computing, the internal network is fixed, so training has two stages: simulate the reservoir to collect internal states, then tune the regularization to optimize the readout weights.

To generate data for readout optimization, we simulate the reservoir’s response to the target signal using **teacher forcing**: the reservoir is driven by the true Lorenz signal. At each time step, all neuron activations are stored in a state matrix R .

During this phase, we discard a washout period of 200 time steps. This removes the transient caused by zero-state initialization, so R reflects synchronized Lorenz-driven dynamics.

Figure 5 shows the post-washout input signal and the activations of 8 sampled neurons. For this Gaussian baseline, we used $N = 250$ neurons, spectral radius $J = 0.8$, and input scaling 0.1. The activations are heterogeneous without saturating, giving the regression model a useful high-dimensional representation.

Once R is collected, the output weights W_{readout} are computed with ridge regression. To learn forward dynamics, the reservoir state at time t predicts the target at $t + 1$; the shifted targets form H_{target} .

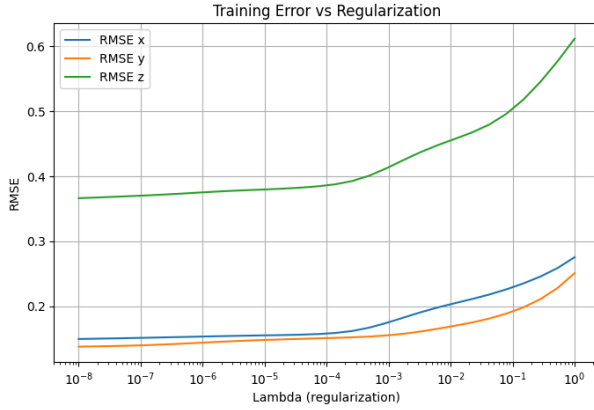
The weights are then obtained by solving a regularized least-squares problem:

$$W_{\text{readout}} = \arg \min_W |WR - H_{\text{target}}|^2 + \lambda |W|^2$$

where λ penalizes large weights and helps prevent overfitting to noise or small fluctuations in the training data.

We also augment the state matrix with a constant bias term to capture spatial offsets without distorting the weights.

The main experimental focus in this phase was tuning λ . On the baseline network ($N = 250$), we tested 30 logarithmically spaced values from 10^{-18} to 1.



Increasing the regularization penalty raises training RMSE across all three spatial axes. The best training fit occurs for very small λ values, around 10^{-8} .

5. Human connectome data and network analysis

We now turn to the human connectome data and examine how its graph structure can serve as a biologically grounded reservoir.

The dataset comes from a preprocessed Human Connectome Project release in which each subject is stored as a .graphml file representing brain structural connectivity.

Each file records brain regions and the connections between them. Nodes correspond to regions, while edges represent structural connections such as white matter fiber tracts.

Graph-theoretically, the connectome is modeled as $G = (V, E)$, where V is the node set and E the edge set. The graph is undirected, so the connectivity matrix is symmetric: $W_{ij} = W_{ji}$.

The analyzed graph has 463 nodes, determined by a brain parcellation that balances anatomical detail with computational cost. This resolution remains manageable while preserving meaningful connectivity structure.

A relatively large node count also makes network statistics clearer. In smaller graphs, finite-size fluctuations can obscure the underlying structure.

Each node includes its 3D position, hemisphere, and anatomical label. Edges describe regional connections and include mean fiber length, fractional anisotropy (FA), and number of fibers.

We separate the graph into node- and edge-level information to analyze both spatial organization and connection structure.

At the node level, positions show a clear 3D brain organization. The distribution is roughly symmetric, with visible separation between hemispheres; most regions form a compact structure, while boundary nodes are more spread out. This confirms that the graph preserves anatomical layout.

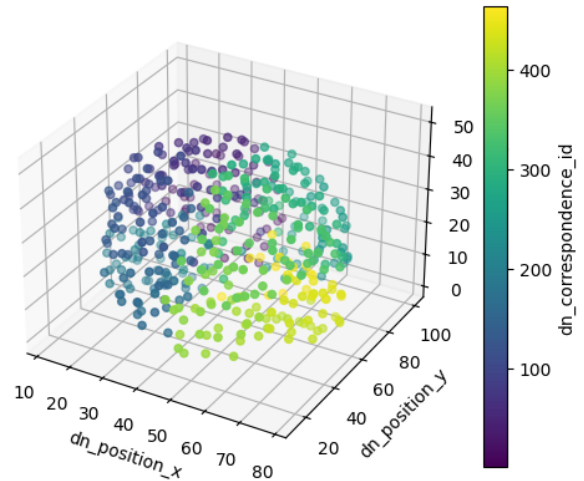


Figure 6: Connectome nodes in 3D, colored by hemisphere.

At the edge level, connection features are strongly heterogeneous. The number of fibers ranges from about 1 to more than 4600, with median around 22, so most connections are weak. The distribution is strongly right-skewed, with many low values and a long tail of high-weight edges.

The high skewness (≈ 6.37) and large kurtosis (≈ 63) support this heavy-tailed interpretation.

Fiber length spans a wider range, roughly 10 to 100, and decays more smoothly, with moderate skewness.

The FA distribution is more concentrated and decreases rapidly as values increase. Visually it

resembles an exponential-like decay, though a formal test would be needed to confirm this.

Overall, the connectome is heterogeneous rather than uniform, with a small subset of connections carrying much larger weights.

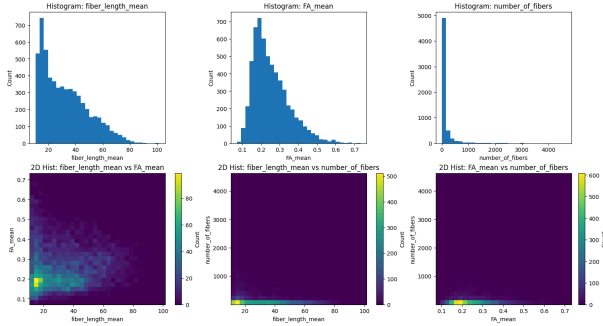


Figure 7: Edge-feature histograms and pairwise densities.

We further examine the edge features through correlation analysis. Most relationships are weak. The clearest pattern is a moderate negative correlation between fiber length and number of fibers ($\rho \approx -0.37$), suggesting that longer connections tend to involve fewer fibers.

Correlations involving FA are much weaker ($\rho \approx 0.08$ with fiber length and $\rho \approx 0.04$ with number of fibers), suggesting that FA captures a different aspect of connectivity.

We use Spearman’s rank correlation rather than Pearson correlation because the edge features, especially fiber count, are strongly skewed. Rank correlation is less dominated by extreme values and is therefore more robust here.

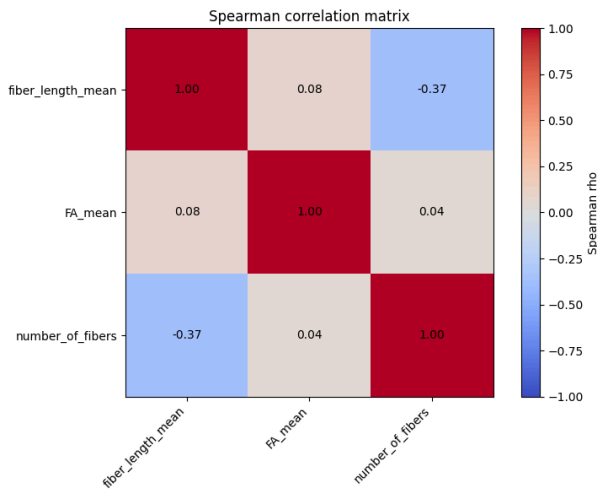


Figure 8: Spearman correlations between structural edge features.

Finally, the network shows non-trivial **community structure**: groups of nodes are more strongly connected internally than with the rest of the graph. This supports the view of the connectome as a structured, not random, network.

5.1. Training and prediction on the Lorenz system

We next introduce the chaotic system used to test the reservoir, chosen for its simplicity and rich nonlinear dynamics.

The Lorenz system is a set of three coupled ordinary differential equations developed by Edward Lorenz while studying atmospheric convection. It is a classic example of deterministic chaos: the equations contain no randomness, yet they are highly sensitive to initial conditions and can display aperiodic behavior.

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

The parameters ρ , σ , and β are constants, while $x(t)$, $y(t)$, and $z(t)$ are time-dependent state variables. Sensitivity to initial conditions makes long-term prediction infeasible. Although the system is deterministic, small numerical errors are amplified exponentially, so trajectories eventually diverge from the true continuation. Another key property is boundedness. Although trajectories are chaotic and sensitive, they do not diverge to infinity; instead, they remain confined to the butterfly-shaped Lorenz attractor.

This makes the system useful for reservoir computing: beyond short-term accuracy, it tests whether the model learns the qualitative attractor geometry. A good autonomous model should remain bounded even after exact phase alignment is lost. We generate the Lorenz attractor by numerical integration with the Euler method, chosen for speed. The classical Lorenz-63 parameters place the system in a chaotic but bounded regime that produces the familiar butterfly attractor.

6. Results

In the final stage of the project, we compared the different reservoir variants on the same Lorenz

prediction task. In each case, the reservoir was first driven by the true Lorenz signal using teacher forcing. After training the linear readout, the model was then run autonomously, meaning that its own predicted output was fed back as the next input.

Because the Lorenz system is chaotic, exact trajectory matching is not expected over long horizons. For this reason, we evaluate the models in two ways: visually, by checking whether the autonomous prediction stays on a bounded Lorenz-like attractor, and quantitatively, using the divergence time t_{div} . This is defined as the first time at which the Euclidean distance between the prediction and the true trajectory exceeds a fixed threshold ε .

6.1. Gaussian random reservoir baseline

The Gaussian reservoir was used as a reference model because its recurrent matrix is dense and randomly generated, rather than based on brain connectivity. In our baseline run, the reservoir used $N = 300$ neurons, target spectral radius $J = 0.85$, input scaling 0.5, leak parameter $\alpha = 1$, and a training window of 10000 time units. The measured spectral radius after scaling was 0.8500, and the full run took about 26.4 seconds.

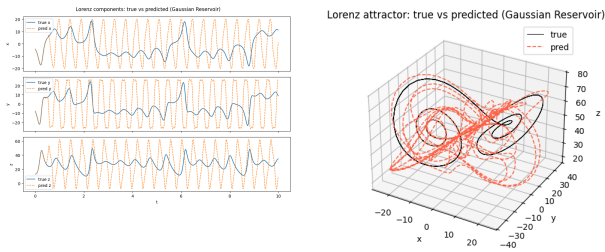


Figure 9: Gaussian reservoir prediction: components and 3D attractor.

The baseline produces a bounded oscillatory prediction, so it does learn part of the Lorenz structure. However, it loses phase alignment with the true trajectory after a short time. In the 3D plot, the predicted path remains in roughly the correct region of phase space, but the loops are more regular and more spread out than the true attractor.

This suggests that the model learned a coarse autonomous oscillator rather than the exact Lorenz flow. The main difficulty is the closed-loop

setting: during training, the model only needs to make one-step predictions, but during autonomous prediction even small errors are fed back into the system. Since the Lorenz dynamics amplify small differences, these errors quickly accumulate.

6.2. Human-connectome reservoir

The main connectome model replaces the Gaussian recurrent matrix with the weighted adjacency matrix of a human structural connectome. This gives the reservoir an anatomical topology while keeping the same basic learning procedure: the internal weights are fixed, and only the linear readout is trained. The experiment used $J = 0.9$, input scaling 0.5, ridge parameter $\lambda = 3e - 6$, and a training horizon of 20000 time units. The run took about 111.0 seconds, with a divergence time of approximately $t_{\text{div}} = 0.39s$ for $\varepsilon = 1.0$.

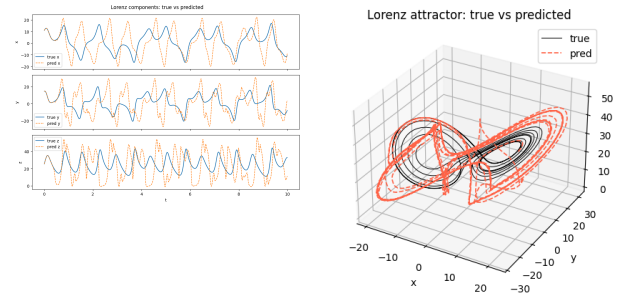


Figure 10: Connectome reservoir prediction: components and 3D attractor.

The component-wise prediction starts at the correct scale and initially follows the oscillatory pattern of the Lorenz signal. After this short initial period, the predicted and true trajectories separate. In the 3D view, the autonomous trajectory remains bounded and still resembles a Lorenz attractor, although it expands away from the true one.

The short divergence time is not surprising, since the Lorenz system is highly sensitive to small errors. The more relevant result is that the connectome-based reservoir still produces nontrivial autonomous dynamics, even though its recurrent structure was not designed for this task. At the same time, the mismatch with the true trajectory shows that an anatomical network is not automatically optimal for predicting an artificial chaotic system.

6.3. Leak-rate connectome reservoir

We also tested a leaky version of the connectome reservoir. In this model, the new reservoir state is mixed with the previous state, which slows down the update and gives the system more inertia. The tested configuration used $\alpha = 0.3$, $J = 0.9$, input scaling 0.5, $\lambda = 3e - 6$, and a shorter training horizon of 1000 time units. This run took about 3.74 seconds.

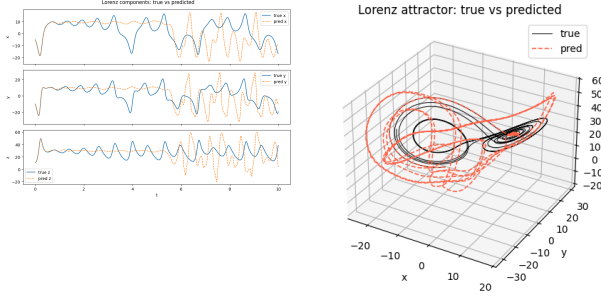


Figure 11: Leaky connectome prediction ($\alpha = 0.3$).

The leaky reservoir gives smoother autonomous predictions. The model drifts away from the true signal more gradually, and the 3D trajectory remains bounded. However, it still does not reproduce the exact lobe-switching behavior of the Lorenz attractor.

This result shows the main trade-off introduced by the leak rate. A smaller α can improve smoothness and memory because the reservoir changes less abruptly from one step to the next. On the other hand, the model may respond too slowly to the fast changes in the input signal. Since this run used a shorter training window, we treat it mainly as a qualitative comparison rather than evidence of a fully optimized improvement.

6.4. Perturbation experiments

To test robustness, we applied four types of perturbations to the connectome reservoir: node deletion, edge deletion, edge swapping, and node-label swapping. For each perturbation level p , we averaged the divergence time across multiple connectome graphs and random seeds.

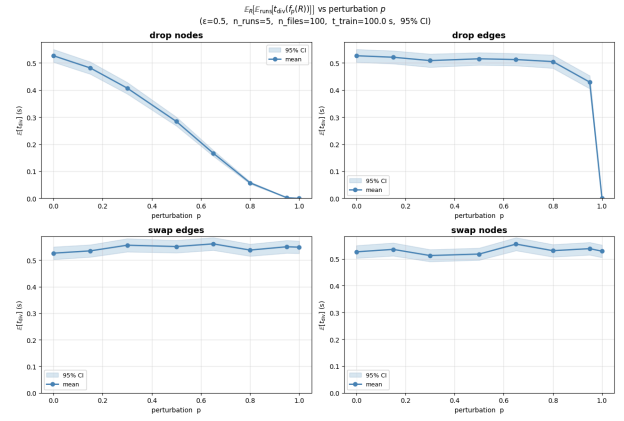


Figure 12: Divergence time under four connectome perturbations.

Node deletion has the strongest effect. As p increases, the average divergence time drops sharply and approaches zero when most nodes are removed. This is expected because deleting nodes reduces the dimension of the reservoir. With fewer internal variables, the readout has less information available to reconstruct the Lorenz state.

Edge deletion has a weaker effect over most of the tested range and becomes clearly damaging only at very high deletion levels. This points to redundancy in the graph: many individual connections can be removed while enough recurrent structure remains for short-term prediction. There is also an implementation effect to consider. After perturbation, each graph is rescaled by its spectral radius, so the overall recurrent gain remains comparable across perturbation levels. This normalization probably reduces the apparent impact of deleting edges.

Edge swapping produces little visible change in the plotted range. For this task, preserving the number of nodes and the global recurrent strength seems more important than preserving the exact anatomical placement of every edge. Node-label swapping also has little effect because it mainly changes the ordering or labels of nodes while leaving the underlying graph structure almost unchanged.

6.5. High-damage edge-deletion sweep

We then examined edge deletion more closely at high perturbation levels. The average divergence time changes only slightly between $p = 0$ and $p =$

0.90, then decreases more noticeably at $p = 0.95$ and $p = 0.99$. In this sweep, the mean divergence time is about $0.51s$ with no edge deletion and about $0.36s$ at $p = 0.99$.

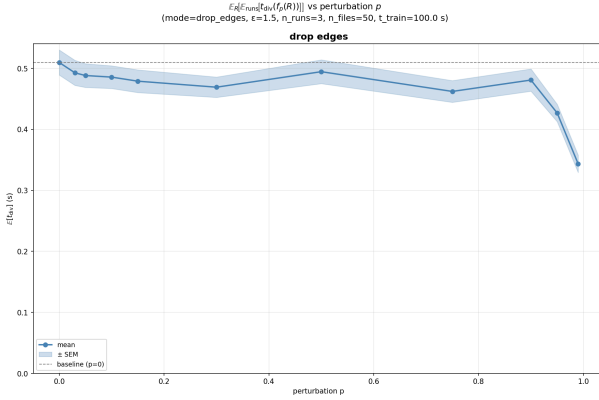


Figure 13: High-damage edge-deletion sweep.

This confirms that the connectome reservoir is fairly robust to moderate edge loss. A possible explanation is that many deleted edges are either weak or redundant, so the remaining graph can still support short-horizon dynamics. At extreme deletion levels, however, too much connectivity is removed, and the reservoir no longer provides a rich enough state representation.

7. Conclusion

Overall, the experiments show that both Gaussian and connectome-based reservoirs can produce bounded Lorenz-like trajectories, but neither gives reliable long-horizon prediction with the tested hyperparameters. This is consistent with the chaotic nature of the Lorenz system. Once the model is run autonomously, small readout errors are repeatedly fed back into the reservoir and are quickly amplified.

The connectome result is still interesting because it shows that a fixed anatomical graph can function as a reservoir and generate nontrivial dynamics without training its internal weights. However, the task also makes clear that biological structure alone does not guarantee accurate prediction of an artificial chaotic signal.

The perturbation experiments give the clearest conclusion. Removing nodes is much more harmful than removing or rewiring edges because it directly reduces the number of reservoir states available to the readout. Edge deletion and edge

swapping have weaker effects, probably because the graph contains redundant pathways and because spectral rescaling preserves the overall recurrent strength. The leak-rate experiment suggests that slower reservoir updates can make predictions smoother, but this does not by itself solve the divergence problem.

A stronger follow-up study would tune J , α , input sparsity, and regularization more systematically. It would also be useful to compare several connectome subjects using the same preprocessing pipeline and to repeat the perturbation experiments both with and without spectral rescaling. This would make it easier to separate the effect of anatomical topology from the effect of global gain.